

IMPACT code challenge

Challenge:

We expect you to invest 2 hours in this Code Challenge; we don't expect you to meet all expectations. We advise you to find a quiet time and space to focus on the Code Challenge so that you can work without interruptions during this period. Use our Code Challenge API to build your own Basket API. Assume your Basket API is going to be used by end-users via a typical online shopping frontend. Therefore, your Basket API should support the regular operations of an online shopping cart, e.g., add product(s) to the basket, remove product(s) from the basket, decrease and increase product quantity, and submit an order using the basket.

Note that the mentioned operations do not correspond directly to endpoints. It is up to you to decide which endpoints make sense in your Basket API. To support the development of your Basket API, we provide our own Code Challenge API. You can have a look at its Swagger UI available here:

<https://azfun-impact-code-challenge-api.azurewebsites.net/api/swagger/ui>

The Code Challenge API will provide a Product Catalog containing 10,000 ranked products. Warning: we will return the whole 10,000 products without pagination. From these 10,000 products, only the top-ranked 100 products should be allowed to be bought online. The Code Challenge API requires authentication. You can obtain a Token with the Login endpoint using your email. However, to simplify your implementation, assume that any clients of your API should not need authentication, hence they should not insert an email address on any of your endpoints. The objective of authentication in the Code Challenge API is to authenticate your API alone.

Lastly, once the end-user has selected the desired products, you should allow them to submit the basket, thus calling our CreateOrder endpoint.

Beyond the endpoints you designed for the Basket API, you should also provide the following endpoints:

- Return the top-ranked 100 products.
- Return paginated results of the product catalog ordered by price in ascending order and properly paginated.
- The page size should be defined by the caller of your API, but your API should not allow page sizes above 1000.
- Get a basket via its Id, which we prefer to be a GUID.
- Get the 10 cheapest products from all the 10,000 products.

AI Tools:

You are encouraged to use AI coding assistants (GitHub Copilot, Cursor, Claude, ChatGPT, or similar) during the challenge. Consider using AI to help you understand the task, sketch your API design, generate boilerplate, write tests, or review your own code. Keep brief notes on what you used AI for — we will ask about it in the interview.

Code delivery:

- It is required to commit your code to a public git repository, such as GitHub or Bitbucket.
- Code submitted with multiple commits, ideally with a first commit once you start the Code Challenge.
- Include a brief README or inline comments explaining key design decisions.

Expectations:

The following are our expectations for the solution you provide:

- We expect to be able to run your solution locally and run the tests you have implemented without any changes to your code.
- A solution designed to adhere to SOLID principles.
- Production-ready code with unit and end-to-end testing.
- You must avoid frontend and/or end-user authentication in your Basket API, also requesting an email address on any of your endpoints.
- We don't expect you to store the baskets in any persistence layer, e.g. database or file. Storing it in memory is enough.